Assignment No: 11

Title: Create a webpage to demonstrate DOM manipulation, event handling.

Aim: To create a webpage that demonstrates Document Object Model (DOM) manipulation - selecting, creating, inserting, modifying and removing nodes - together with event handling, event objects, event propagation and event delegation.

Objectives:

1. To understand the DOM as a tree representation of an HTML document.
2. To select elements using getElementById, querySelector and querySelectorAll.
3. To create and insert nodes using createElement, appendChild and the modern insertion methods.
4. To modify content and attributes using textContent, innerHTML, setAttribute and classList.
5. To remove nodes from the document.
6. To register and remove event handlers and to read the event object.
7. To understand event bubbling, capturing and delegation.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS

Theory:

1. The Document Object Model

When a browser parses an HTML document it builds the DOM: a tree of node objects in which every element, attribute and piece of text is a node. JavaScript can read and change this tree at runtime, and the browser re-renders the page to match. The global document object is the entry point to the tree.

2. Selecting Elements

Method	Returns
document.getElementById(id)	One element, or null
document.querySelector(css)	The first match of a CSS selector, or null
document.querySelectorAll(css)	A static NodeList of all matches
document.getElementsByClassName(c)	A live HTMLCollection
element.closest(css)	The nearest ancestor matching the selector

3. Creating, Inserting and Removing Nodes

Method	Purpose
document.createElement(tag)	Creates a detached element
document.createTextNode(text)	Creates a text node
parent.appendChild(node)	Adds the node as the last child
parent.append(...nodes)	Adds nodes or strings at the end
parent.insertBefore(new, ref)	Inserts before an existing child
node.remove()	Removes the node from its parent
parent.replaceChild(new, old)	Swaps one child for another

4. Modifying Content, Attributes and Styles

Property / Method	Effect
textContent	Reads or writes plain text; safe against HTML injection
innerHTML	Reads or writes markup; unsafe with untrusted input
setAttribute / getAttribute	Reads or writes any attribute
classList.add / remove / toggle	Manipulates the class attribute
element.style.property	Sets an inline CSS declaration
dataset.name	Reads or writes a data-name attribute

5. Event Handling and the Event Object

addEventListener(type, handler, options) registers a handler; removeEventListener removes it. The handler receives an event object carrying e.type, e.target (the element that fired the event), e.currentTarget (the element the handler is attached to), e.key for keyboard events and e.dataTransfer for drag-and-drop.

- e.preventDefault() cancels the browser's default action, such as following a link or opening a dropped file.
- e.stopPropagation() stops the event travelling further up the tree.
- { once: true } removes the handler automatically after it fires once.

6. Propagation and Event Delegation

An event travels down from the document to the target (capturing phase) and then back up again (bubbling phase). Handlers run in the bubbling phase by default. Event delegation exploits this: one handler is attached to a container and uses e.target to decide which child was acted upon. This works for elements added later and uses one listener instead of hundreds.

Delegation pattern:

```
list.addEventListener('click', (e) => {
  const row = e.target.closest('li');
  if (!row) return;
  console.log('clicked', row.dataset.id);
```

```
});
```

7. Dynamically Loading a Script

A third-party library can be loaded on demand by creating a <script> element, setting its src and appending it to document.head. The library runs once the browser has downloaded it, which is how LipiSutra loads the Google Maps SDK only when a user opens the Origin tab.

Course Project Implementation:

Course-Project Mapping: Available in the course project. Submit src/components/MapSection.jsx as the primary file - it creates a <script> node with document.createElement, appends it to document.head, builds InfoWindow markup as an HTML string and registers marker.addListener handlers. Submit src/components/UploadSection.jsx alongside it for direct DOM access with document.getElementById(...).click(), drag-and-drop event handling with preventDefault and dataTransfer, and the FileReader event API.

Concept	File	Evidence
document.createElement + appendChild	src/components/MapSection.jsx	script = document.createElement("script"); document.head.appendChild(script)
document.getElementById	src/components/MapSection.jsx, UploadSection.jsx	getElementById("heritage-map"), getElementById("file-input").click()
Building markup dynamically	src/components/MapSection.jsx	InfoWindow content string with interpolated values
addEventListener-style handlers	src/components/MapSection.jsx	marker.addListener("click", () => info.open(map, marker))
preventDefault + dataTransfer	src/components/UploadSection.jsx	onDragOver={e => e.preventDefault()}, e.dataTransfer.files[0]
FileReader events	src/components/UploadSection.jsx	reader.onload = () => setBase64(...); reader.readAsDataURL(file)
Programmatic click	src/components/UploadSection.jsx	a styled div forwarding its click to a hidden file input

Program:

Project file - src/components/MapSection.jsx (node creation, head injection, listeners)

```
import { useEffect } from "react";

const GOOGLE_KEY = import.meta.env.VITE_MAPS_KEY || import.meta.env.VITE_VISION_KEY;

export default function MapSection({ locations, theme }) {
  useEffect(() => {
    const initMap = () => {
```

```
      if (!window.google) return;

      // SELECT an existing node by id and hand it to the SDK
      const map = new window.google.maps.Map(
        document.getElementById("heritage-map"),
        { center: { lat: 20.5937, lng: 78.9629 }, zoom: 5, disableDefaultUI: true }
      );

      if (locations && locations.length > 0) {
        const geocoder = new window.google.maps.Geocoder();
        const bounds   = new window.google.maps.LatLngBounds();

        locations.forEach(loc => {
          geocoder.geocode({ address: loc }, (results, status) => {
            if (status !== 'OK' || !results[0]) {
              console.error(`[GEOCODER] Failed to find '${loc}': ${status}`);
              return;
            }

            const marker = new window.google.maps.Marker({
              position: results[0].geometry.location,
              map, title: loc
            });

            // BUILD MARKUP DYNAMICALLY from data
            const info = new window.google.maps.InfoWindow({
              content: `<div style="background:#131A2B;color:#F8F9FA;padding:16px;
                                    border-radius:12px">
                          <strong style="color:#D4AF37">${loc}</strong>
                          <span style="font-size:12px;display:block">
                            Identified from document analysis
                          </span>
                        </div>`
            });

            // EVENT HANDLER registered on the created object
            marker.addListener("click", () => info.open(map, marker));

            bounds.extend(results[0].geometry.location);
            map.fitBounds(bounds);
          });
        });
      }
    };

    if (window.google) {
      initMap();
    } else {
      // CREATE a <script> node and INSERT it into <head> on demand
      const script  = document.createElement("script");
      script.src    = `https://maps.googleapis.com/maps/api/js?key=${GOOGLE_KEY}`
                    + `&callback=initLipisutraMap`;
      script.async  = true;
      window.initLipisutraMap = initMap;        // the SDK calls this when ready
      document.head.appendChild(script);
    }
  }, [locations]);

  return (
    <div className="rounded-xl overflow-hidden border"
         style={{ backgroundColor: theme.headerBg, borderColor: theme.border }}>
      <div className="p-5 border-b">
        <h2 style={{ color: theme.accent }}>Heritage Map</h2>
        <p className="text-xs">
          {locations && locations.length > 0
            ? `Displaying ${locations.length} origin sites via AI Auto-Pin.`
            : "Atlas overview of historical records."}
        </p>
      </div>
```

```
      {/* the container the SDK will fill - selected above by this id */}
      <div id="heritage-map" className="w-full h-[400px]"
           style={{ backgroundColor: theme.bg }} />
    </div>
  );
}
```

Project file - src/components/UploadSection.jsx (drag-and-drop and programmatic click)

```
function handleFileDrop(e) {
  e.preventDefault();                                  // stop the browser opening
the file

  // a drop carries dataTransfer; a picker change carries target.files
  const file = e.dataTransfer ? e.dataTransfer.files[0] : e.target.files[0];
  if (!file) return;

  setImage(URL.createObjectURL(file));                 // object URL for the preview

  const reader = new FileReader();
  reader.onload = () => setBase64(reader.result.split(",")[1]);   // EVENT on
FileReader
  reader.readAsDataURL(file);

  onResult(null);
}

// ... in the JSX:
<div
  onDragOver={e => e.preventDefault()}                 // required for a drop to fire
  onDrop={handleFileDrop}
  onClick={() => document.getElementById("file-input").click()}   // PROGRAMMATIC
CLICK
>
  <h2>Upload Artifact Image</h2>
  <p>Drag and drop a high-resolution image, or browse files.</p>

  {/* the real control is hidden; the styled div forwards the click to it */}
  <input type="file" id="file-input" accept="image/*"
         onChange={handleFileDrop} className="hidden" />
</div>
```

Supplementary standalone demonstration - assignment11.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LipiSutra - DOM Manipulation</title>
  <style>
    body    { font-family: Georgia, serif; background:#f5ede0; color:#1a0800;
padding:24px; }
    #shelf  { list-style:none; padding:0; max-width:620px; }
    #shelf li { background:#fff; border:1px solid #8B4513; border-radius:8px;
                padding:12px; margin-bottom:8px; display:flex; justify-content:space-
between; }
    #shelf li.verified { background:#e6f4ea; border-color:#2e7d32; }
    #dropzone { border:2px dashed #8B4513; border-radius:12px; padding:40px;
                text-align:center; margin:20px 0; max-width:620px; }
    #dropzone.hot { background:#ede0cc; border-style:solid; }
    button  { margin-right:8px; }
  </style>
</head>
<body>
  <h1>LipiSutra Archive Shelf</h1>
```

```
  <input id="newScript" type="text" placeholder="Script name, e.g. Sharada">
  <button id="addBtn">Add Manuscript</button>
  <button id="clearBtn">Clear Shelf</button>

  <ul id="shelf"></ul>

  <div id="dropzone">Drop an artifact image here</div>
  <img id="preview" alt="" style="display:none; max-width:260px;
       border:1px solid #8B4513; border-radius:8px;">

  <p id="log"></p>

  <script src="assignment11.js" defer></script>
</body>
</html>
```

Supplementary standalone demonstration - assignment11.js

```
"use strict";

// ---------- 1. SELECTING NODES ----------
const shelf    = document.getElementById("shelf");
const input    = document.getElementById("newScript");
const dropzone = document.getElementById("dropzone");
const preview  = document.getElementById("preview");
const log      = document.getElementById("log");

let counter = 100;

// ---------- 2. CREATING AND INSERTING NODES ----------
function addManuscript(scriptName) {
  counter += 1;

  const li = document.createElement("li");          // create
  li.dataset.id = "M-" + counter;                   // data-* attribute
  li.setAttribute("title", "Click to toggle verification");

  const label = document.createElement("span");     // nested child
  label.textContent = `M-${counter} - ${scriptName}`;   // safe text insertion

  const del = document.createElement("button");
  del.textContent = "Remove";
  del.className   = "remove-btn";

  li.appendChild(label);                            // insert
  li.appendChild(del);
  shelf.appendChild(li);

  log.textContent = `Added M-${counter}. Shelf now holds ` +
                    `${shelf.querySelectorAll("li").length} manuscripts.`;
}

// ---------- 3. EVENT HANDLING : click ----------
document.getElementById("addBtn").addEventListener("click", () => {
  const value = input.value.trim();
  if (value === "") { log.textContent = "Type a script name first."; return; }
  addManuscript(value);
  input.value = "";
  input.focus();
});

// ---------- 4. EVENT HANDLING : keydown on an input ----------
input.addEventListener("keydown", (e) => {
  if (e.key === "Enter") document.getElementById("addBtn").click();
});
```

```
// ---------- 5. EVENT DELEGATION : one handler for every current and future row
----------
shelf.addEventListener("click", (e) => {
  const row = e.target.closest("li");
  if (!row) return;                                  // clicked the empty area

  if (e.target.classList.contains("remove-btn")) {
    e.stopPropagation();                             // do not also toggle the row
    row.remove();                                    // REMOVING A NODE
    log.textContent = `Removed ${row.dataset.id}.`;
    return;
  }

  row.classList.toggle("verified");                  // MODIFYING CLASSES
  const state = row.classList.contains("verified") ? "verified" : "pending";
  log.textContent = `${row.dataset.id} marked ${state}. ` +
                    `(target=${e.target.tagName}, currentTarget=$
{e.currentTarget.id})`;
});

// ---------- 6. CLEARING : removing many nodes ----------
document.getElementById("clearBtn").addEventListener("click", () => {
  while (shelf.firstChild) shelf.removeChild(shelf.firstChild);
  log.textContent = "Shelf cleared.";
});

// ---------- 7. DRAG AND DROP with preventDefault ----------
dropzone.addEventListener("dragover", (e) => {
  e.preventDefault();                                // required or 'drop' never fires
  dropzone.classList.add("hot");
});
dropzone.addEventListener("dragleave", () => dropzone.classList.remove("hot"));
dropzone.addEventListener("drop", (e) => {
  e.preventDefault();                                // stop the browser opening the
file
  dropzone.classList.remove("hot");

  const file = e.dataTransfer.files[0];
  if (!file || !file.type.startsWith("image/")) {
    log.textContent = "Drop an image file.";
    return;
  }
  preview.src = URL.createObjectURL(file);
  preview.style.display = "block";                   // MODIFYING INLINE STYLE
  log.textContent = `Loaded ${file.name} (${(file.size / 1024).toFixed(0)} KB).`;
});

// ---------- 8. A ONE-SHOT LISTENER ----------
window.addEventListener("DOMContentLoaded", () => {
  ["Modi", "Devanagari", "Brahmi"].forEach(addManuscript);
  log.textContent = "Shelf initialised with three manuscripts.";
}, { once: true });
```

Output:

- On load, three list items are created with createElement and appended to the shelf.
- Typing "Sharada" and pressing Enter triggers the keydown handler, which forwards to the Add button and inserts a fourth row.
- Clicking a row toggles its verified class and logs "M-101 marked verified. (target=SPAN, currentTarget=shelf)" - showing that the handler lives on the container while the event originated on a child.

- Clicking Remove deletes that node only; stopPropagation prevents the row from also being toggled.
- Rows added after the page loaded respond to clicks without any new listener, proving that delegation works for dynamically created nodes.
- Dragging an image over the dropzone adds the hot class and turns the border solid; dropping it shows a preview and logs the file name and size.
- In the running LipiSutra application, opening the Origin tab injects the Google Maps script into <head> at that moment - visible as a new entry in the Network tab - and clicking a gold marker opens its InfoWindow.

Conclusion:

In this assignment, we demonstrated DOM manipulation and event handling. We selected nodes by id and CSS selector, created and inserted elements with createElement and appendChild, modified text, attributes, classes and inline styles, and removed nodes from the tree. We registered handlers with addEventListener, read e.target and e.currentTarget from the event object, used preventDefault and stopPropagation, and applied event delegation so that a single listener serves rows created after page load. The LipiSutra project applies the same techniques to inject the Maps SDK on demand and to drive its drag-and-drop upload.

References:

8. Mozilla Developer Network (MDN) Web Docs - JavaScript Guide, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
9. W3Schools - JavaScript Tutorial, https://www.w3schools.com/js/
10. ECMAScript Language Specification, https://tc39.es/ecma262/
11. MDN Web Docs - Document Object Model (DOM), https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
12. MDN Web Docs - EventTarget.addEventListener(), https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener